

JavaScript: tipos, conversões e estruturas condicionais

typeof • tipos • conversões • igualdade • condicionais

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de:

- Utilizar typeof para verificar o tipo de um valor.
- Reconhecer os principais tipos de valores em JavaScript.
- Compreender conversões implícitas e realizar conversões explícitas quando necessário.
- Diferenciar igualdade com conversão (==) de igualdade restrita (===).
- Compreender valores truthy e falsy em expressões condicionais.
- Utilizar if, else, else if, switch e o operador condicional ?: em decisões.
- Reconhecer usos condicionais de &&, || e ?? em expressões simples.

O operador typeof

O operador typeof permite verificar o tipo associado ao valor avaliado por uma expressão.

```
let idade = 20;  
let nome = "Ana";  
let aprovado = true;  
  
console.log(typeof idade);  
console.log(typeof nome);  
console.log(typeof aprovado);
```

Saída no console

```
number  
string  
boolean
```

typeof será usado durante a aula para observar as conversões de tipo.

Principais tipos de valores

JavaScript trabalha com valores primitivos e objetos. Nesta aula, o foco está nos tipos mais comuns em expressões e entradas de dados.

number

10, 3.14, -5
Números inteiros e reais.

string

"JavaScript"
Texto.

boolean

true, false
Valores lógicos.

undefined

let valor;
Valor ainda não definido.

object

{}, [], null*
Objetos, arrays e o caso histórico de null.

function

function() {}
Funções podem ser tratadas como valores.

Arrays e objetos serão aprofundados em aulas posteriores.

Tipagem dinâmica

Em JavaScript, uma variável pode receber valores de tipos diferentes ao longo da execução.

```
let valor = 10;  
console.log(typeof valor); // number  
  
valor = "dez";  
console.log(typeof valor); // string  
  
valor = true;  
console.log(typeof valor); // boolean
```

Comparação com Java

Em Java, o tipo é declarado com a variável e restringe os valores que podem ser atribuídos.

Em JavaScript, o tipo é determinado pelo valor.

Entrada de dados e tipo

Quando `prompt()` é utilizado no navegador, o valor retornado é uma string.

```
let idade = prompt("Digite sua idade:");  
  
console.log(idade);  
console.log(typeof idade);
```

Exemplo

Se o usuário digitar 20:

idade contém "20"
typeof idade → "string"

Quando a entrada será usada em cálculo, pode ser necessário convertê-la para número.

Conversão de tipos

Conversão de tipo é a transformação de um valor de um tipo para outro. Em JavaScript, ela pode ocorrer de forma implícita ou explícita.

Conversão implícita

A própria linguagem realiza a conversão durante a avaliação de uma expressão.

```
"10" + 5    // "105"  
"10" * 5    // 50
```

Conversão explícita

O programa indica de forma direta qual conversão deve ocorrer.

```
Number("10") // 10  
String(25)   // "25"
```

Conversão para números

As funções abaixo são formas comuns de converter valores textuais recebidos pelo programa.

Number()

Converte o valor para o tipo number.

`Number("25") → 25`

`Number("3.5") → 3.5`

parseInt()

Obtém um número inteiro a partir do início de uma string.

`parseInt("25.8") → 25`

parseFloat()

Obtém um número com parte decimal a partir de uma string.

`parseFloat("25.8") → 25.8`

```
let idade = Number(prompt("Idade:"));
```

Para a maior parte dos cálculos da disciplina, `Number()` será suficiente.

Quando a conversão não produz um número

Se uma conversão numérica não puder produzir um número significativo, o resultado pode ser NaN (Not-a-Number).

```
let valor = Number("JavaScript");  
  
console.log(valor);  
console.log(typeof valor);  
console.log(Number.isNaN(valor));
```

Saída

```
NaN  
number  
true
```

`Number.isNaN(valor)` permite verificar diretamente se o resultado é NaN.

Igualdade: == e ===

Os operadores relacionais já foram apresentados. Em JavaScript, a igualdade merece atenção por causa das conversões de tipo.

== Igualdade

Pode realizar conversões de tipo antes de comparar os valores.

```
5 == "5" // true
```

=== Igualdade restrita

Compara sem realizar a conversão utilizada pelo operador ==.

```
5 === "5" // false
```

Nos exemplos da disciplina, prefira === quando a intenção for comparar sem conversão de tipo.

Valores truthy e falsy

Quando uma expressão é usada em um contexto booleano, JavaScript pode converter seu valor para true ou false.

Valores falsy

false
0
"" (string vazia)
null
undefined
NaN

Outros valores

De modo geral, outros valores são avaliados como truthy em um contexto booleano.

Exemplos: 1, -1, "texto", [], {}

```
let nome = "";  
if (nome) console.log("Nome informado");
```

Estruturas if e if/else

if

Executa um bloco quando a condição é avaliada como verdadeira.

```
let idade = 20;

if (idade >= 18) {
  console.log("Maior de idade");
}
```

if/else

Escolhe entre dois caminhos: um para condição verdadeira e outro para falsa.

```
let nota = 7;

if (nota >= 6) {
  console.log("Aprovado");
} else {
  console.log("Reprovado");
}
```

Múltiplas condições com else if

Uma cadeia de else if permite testar condições em sequência. O primeiro bloco cuja condição for verdadeira é executado.

```
let nota = 8;

if (nota >= 9) {
  console.log("Excelente");
} else if (nota >= 7) {
  console.log("Bom");
} else if (nota >= 6) {
  console.log("Regular");
} else {
  console.log("Reprovado");
}
```

Ordem importa

As condições são avaliadas de cima para baixo.

Coloque primeiro as condições mais específicas quando houver sobreposição.

Condições aninhadas e operadores lógicos

Uma decisão pode aparecer dentro de outra. Em situações simples, operadores lógicos podem reduzir o aninhamento.

Condição aninhada

```
if (idade >= 18) {  
    if (possuiIngresso) {  
        console.log("Entrada permitida");  
    }  
}
```

Com operador lógico

```
if (idade >= 18 && possuiIngresso) {  
    console.log("Entrada permitida");  
}
```

As duas formas podem ser corretas. A escolha depende da lógica que precisa ser representada.

Seleção com switch

A instrução switch seleciona um bloco de código com base no valor de uma expressão.

```
let opcao = 2;

switch (opcao) {
  case 1:
    console.log("Cadastrar");
    break;
  case 2:
    console.log("Consultar");
    break;
  case 3:
    console.log("Excluir");
    break;
  default:
    console.log("Opção inválida");
}
```

Elementos principais

case: valor a comparar.

break: encerra o case.

default: executado quando nenhum case corresponde.

switch sem break: fall-through

Sem break, a execução continua nos cases seguintes até encontrar um break ou terminar o switch.

```
let numero = 1;

switch (numero) {
  case 1:
    console.log("Um");
  case 2:
    console.log("Dois");
}
```

Saída

Um
Dois

O fall-through pode ser intencional, mas deve ser usado de forma clara.

Operador condicional ternário

O operador condicional ?: escolhe entre duas expressões com base em uma condição.

```
condicao ? valorSeVerdadeiro : valorSeFalso
```

```
let idade = 20;  
  
let resultado = idade >= 18  
  ? "Maior de idade"  
  : "Menor de idade";
```

Uso recomendado

Adequado para decisões simples que produzem um valor.

Para lógicas longas ou com várias etapas, prefira estruturas condicionais.

Curto-circuito com &&

No operador &&, a expressão à direita só precisa ser avaliada quando o valor da esquerda não determina um resultado falsy.

```
let logado = true;  
  
logado &&  
  console.log("Usuário autenticado");
```

Leitura prática

Quando a condição à esquerda é truthy, a expressão da direita é avaliada.

Em códigos simples, isso pode ser usado como execução condicionada.

Não substitua automaticamente todo if por &&. Use quando a expressão permanecer clara.

Valores alternativos: || e ??

Operador ||

Pode ser usado para selecionar um valor alternativo quando o primeiro operando é falsy.

```
let nome = "";  
let exibido = nome || "Visitante";  
// "Visitante"
```

Operador ??

Seleciona o valor alternativo apenas quando o primeiro operando é null ou undefined.

```
let quantidade = 0;  
let valor = quantidade ?? 10;  
// 0
```

Diferença importante: 0 e "" são falsy para ||, mas não acionam o valor alternativo de ??.

Formas de decisão vistas na aula

if	Executar um bloco quando uma condição for verdadeira.
if / else	Escolher entre dois caminhos.
else if	Testar várias condições em sequência.
switch	Selecionar entre valores conhecidos.
?:	Produzir um valor com uma decisão simples.
&&	Avaliar a expressão seguinte de forma condicionada.
 	Fornecer alternativa baseada em truthy/falsy.
??	Fornecer alternativa para null ou undefined.

Exemplo integrado: compra com desconto

O exemplo reúne entrada, conversão, validação, condição e operador ternário.

```
let preco = Number(prompt("Preço do produto:"));
let quantidade = Number(prompt("Quantidade:"));

if (Number.isNaN(preco) || Number.isNaN(quantidade)) {
  console.log("Dados inválidos");
} else {
  let total = preco * quantidade;
  let desconto = total >= 200 ? total * 0.10 : 0;
  let valorFinal = total - desconto;

  console.log("Total:", total);
  console.log("Desconto:", desconto);
  console.log("Valor final:", valorFinal);
}
```

Conceitos usados

Number()
Number.isNaN()
||
if / else
operador ?:
operações aritméticas

Prática de fixação

Exercício 1 — Classificação de idade

Receba a idade com `prompt()`, converta para número e classifique em quatro faixas: criança, adolescente, adulto e idoso.

Valide também uma entrada que não possa ser convertida em número.

Exercício 2 — Menu de operações

Receba dois números e uma opção: 1 somar, 2 subtrair, 3 multiplicar, 4 dividir.

Use `switch` para selecionar a operação e trate opção inválida.

Conteúdos: `typeof` • tipos • conversões • `==` e `===` • `truthy/falsy` • `if/else` • `switch` • `?:` • `&&` • `||` • `??`